VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Microprocessors-Microcontrollers

Laboratory Report 01

LED Animations

Advisor(s): Mr. Phan Van Sy
Student(s): Tran Vu Phuong Lam 2452661

HO CHI MINH CITY, SEPTEMBER 2026

Contents

1 Introduction **4**

2 Development Environment **4**

3 Exercise 01: LED Control **4**

3.1 Objective 4

3.2 Implementation 4

4 Exercise 02: Traffic Light **6**

5 Exercise 03: Four-Way Traffic Light **7**

6 Exercise 04: Seven-Segment Display **9**

7 Exercise 05: Traffic Countdown **11**

8 Exercise 06: Analog Clock LED Test **13**

9 Exercise 07: Clear All Clock LEDs **14**

10 Exercise 08: Set a Clock Position **14**

11 Exercise 09: Clear a Clock Position **15**

12 Exercise 10: Integrated Analog Clock **15**

13 Conclusion **16**

List of Figures

3.1 Exercise 01 Proteus schematic and project files 5

4.1 Exercise 02 traffic-light schematic and project files 6

5.1 Exercise 03 four-way traffic-light schematic and project files 7

6.1 Exercise 04 seven-segment schematic and project files 10

8.1 Exercise 06 twelve-LED clock schematic and project files 14

List of Tables

5.1 Traffic light states 7

Listings

3.1 Exercise 01 infinite loop 5
4.1 Exercise 02 traffic-light loop 6
5.1 Exercise 03 initial setup 7
5.2 Exercise 03 control loop 8
6.1 Exercise 04 seven-segment display function 10
7.1 Exercise 05 helper function 11
7.2 Exercise 05 traffic-light countdown loop 11
8.1 Exercise 06 sequential LED test 14
9.1 Exercise 07 clearAllClock function 14
10.1 Exercise 08 setNumberOnClock function 15
11.1 Exercise 09 clearNumberOnClock function 15
12.1 Exercise 10 clock update loop 15

1 Introduction

This report documents the ten exercises specified in Lab 01. The implementations use an STM32F103 microcontroller, the STM32 HAL library, and Proteus simulation. Only the source code relevant to each exercise is included; automatically generated initialization and system code will be omitted.

2 Development Environment

- Microcontroller: STM32F103
- Programming framework: STM32 HAL
- Development environment: STM32CubeIDE
- Compiler toolchain: Arm GNU Toolchain
- Simulation: Proteus
- Project repository: Microprocessors-Microcontrollers-Lab

The repository contains the complete STM32 source code and Proteus design files used for all exercises in this report.

3 Exercise 01: LED Control

3.1 Objective

A yellow LED is connected to PA6 in addition to the red LED on PA5. The two LED states alternate every two seconds.

3.2 Implementation

We use TogglePin to turn it on / off without actually knowing if it is active-low or active-high, it gets the job done, while keeping our project easier to read.

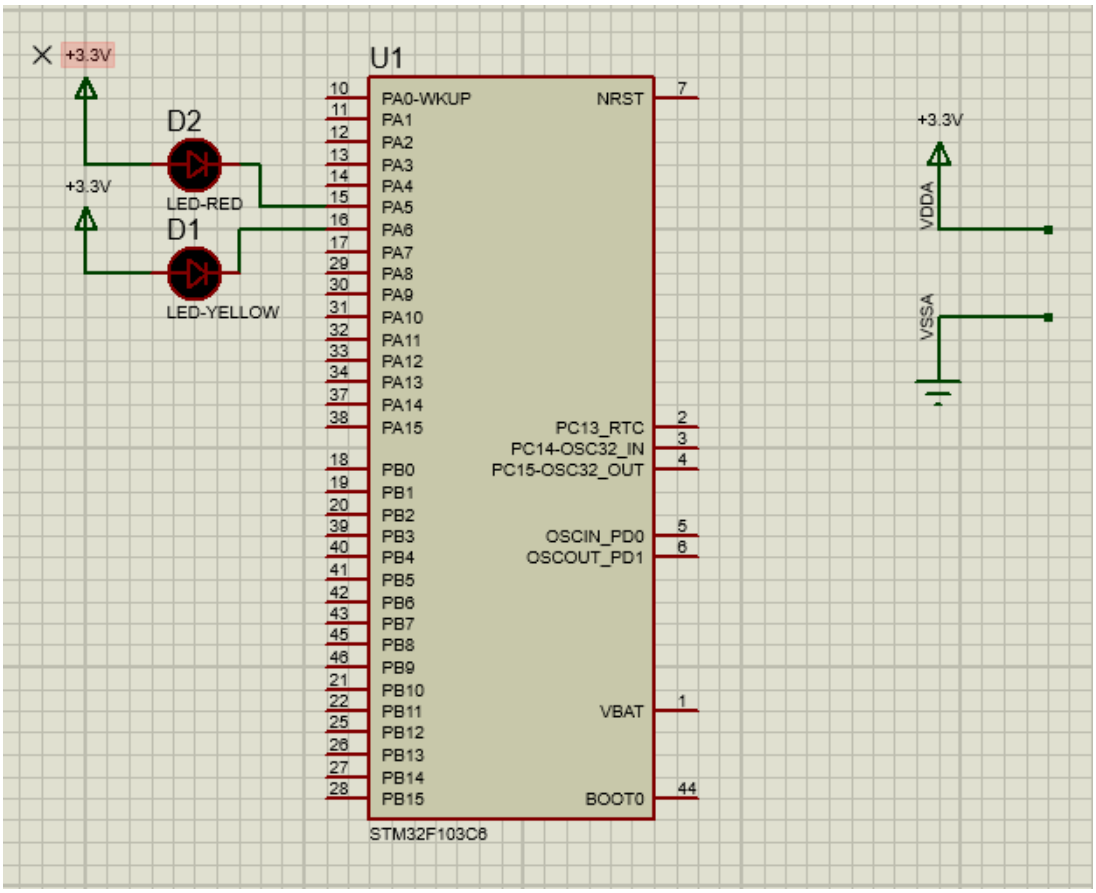


Figure 3.1: Exercise 01 Proteus schematic and project files

Listing 3.1: Exercise 01 infinite loop

```
88  /* USER CODE BEGIN 2 */
89  HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
90  HAL_Delay(2000);
91  /* USER CODE END 2 */
92  /* Infinite loop */
93  /* USER CODE BEGIN WHILE */
94  while (1) {
95    HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
96    HAL_GPIO_TogglePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin);
97    HAL_Delay(2000);
98    /* USER CODE END WHILE */
99  }
```

4 Exercise 02: Traffic Light

Exercise 01 is extended with a green LED connected to PA7. The traffic light uses red, green, and yellow intervals of 5, 2, and 3 seconds, respectively. All three LEDs are controlled through their negative pins.

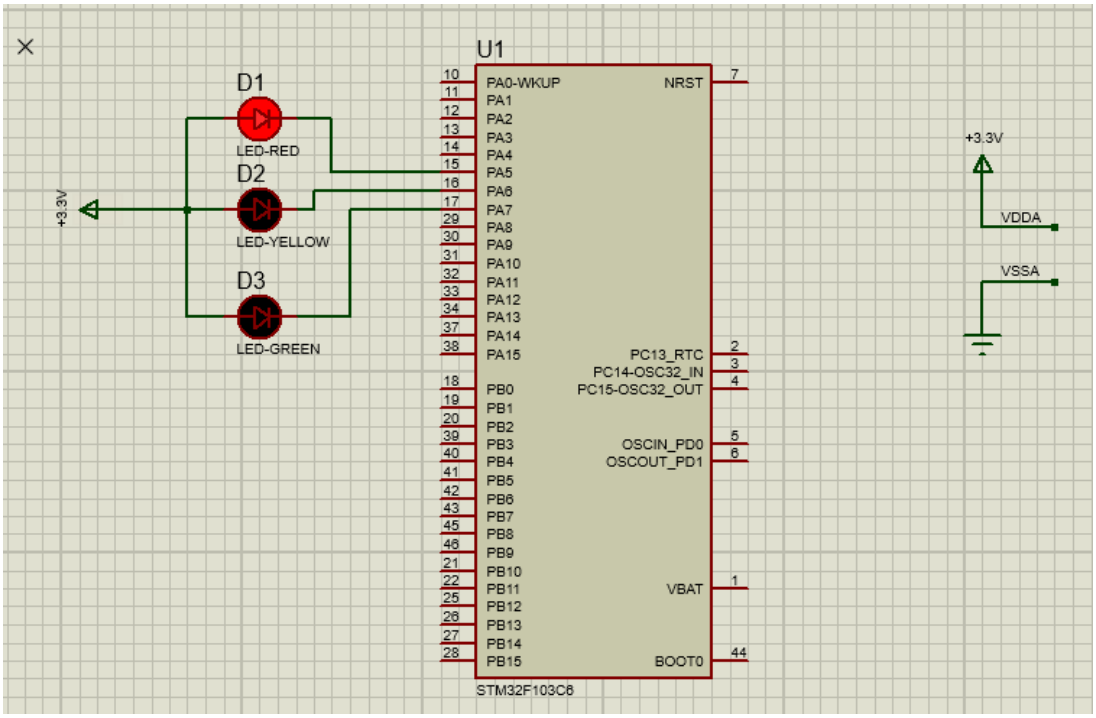


Figure 4.1: Exercise 02 traffic-light schematic and project files

Listing 4.1: Exercise 02 traffic-light loop

```
94  while (1) {
95    /* USER CODE END WHILE */
96    TogglePinDelay(LED_RED_GPIO_Port, LED_RED_Pin, 5000);
97    TogglePinDelay(LED_GREEN_GPIO_Port, LED_GREEN_Pin, 2000);
98    TogglePinDelay(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin, 3000);
99    /* USER CODE BEGIN 3 */
100 }
101 /* USER CODE END 3 */
```

5 Exercise 03: Four-Way Traffic Light

We now quadruple exercise 2 to create a realistic traffic light model for real life use. Althought there are 4 times the traffic light, we only have to care about 2 sessions: Vertical and Horizontal

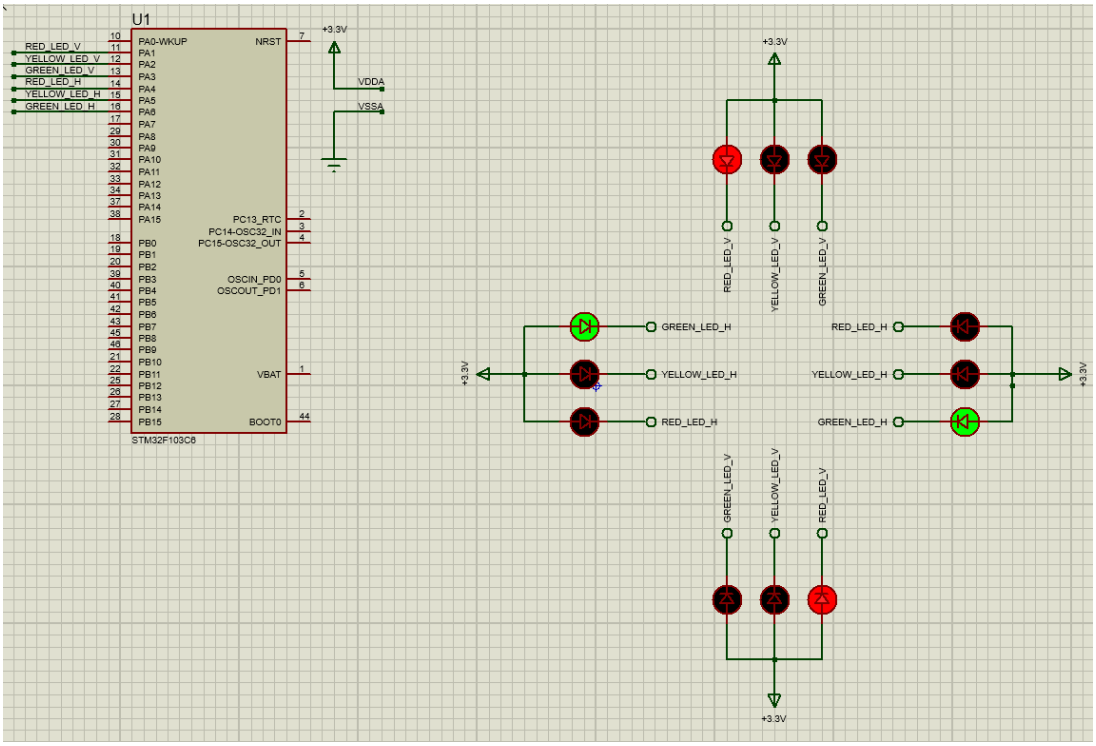


Figure 5.1: Exercise 03 four-way traffic-light schematic and project files

Vertical	Horizontal
Green	Red
Yellow	Red
Red	Green
Red	Yellow

Table 5.1: Traffic light states

We will first let Vertical have the green light, then just swap the states as Table 5.1 for smooth transitions.

Listing 5.1: Exercise 03 initial setup

```
87    MX_GPIO_Init();
88    /* USER CODE BEGIN 2 */
89    HAL_GPIO_WritePin(LED_GREEN1_GPIO_Port,  LED_GREEN1_Pin,
GPIO_PIN_RESET); // ON
90    HAL_GPIO_WritePin(LED_YELLOW1_GPIO_Port, LED_YELLOW1_Pin,
GPIO_PIN_SET);   // OFF
91    HAL_GPIO_WritePin(LED_RED1_GPIO_Port,    LED_RED1_Pin,
GPIO_PIN_SET);   // OFF
92    HAL_GPIO_WritePin(LED_GREEN2_GPIO_Port,  LED_GREEN2_Pin,
GPIO_PIN_SET);   // OFF
93    HAL_GPIO_WritePin(LED_YELLOW2_GPIO_Port, LED_YELLOW2_Pin,
GPIO_PIN_SET);   // OFF
94    HAL_GPIO_WritePin(LED_RED2_GPIO_Port,    LED_RED2_Pin,
GPIO_PIN_RESET); // ON
95    HAL_Delay(5000);
96    /* USER CODE END 2 */
```

Listing 5.2: Exercise 03 control loop

```
101   while (1)
102   {
103     /* USER CODE END WHILE */
104     HAL_GPIO_TogglePin(LED_GREEN1_GPIO_Port,  LED_GREEN1_Pin);  //
Turns OFF
105     HAL_GPIO_TogglePin(LED_YELLOW1_GPIO_Port, LED_YELLOW1_Pin); //
Turns ON
106     HAL_Delay(2000);
107         /* Phase 3: Switch Direction 1 to RED, Direction 2 to GREEN
*/
108     HAL_GPIO_TogglePin(LED_YELLOW1_GPIO_Port, LED_YELLOW1_Pin); //
Turns OFF
109     HAL_GPIO_TogglePin(LED_RED1_GPIO_Port,    LED_RED1_Pin);    //
Turns ON
110     HAL_GPIO_TogglePin(LED_RED2_GPIO_Port,    LED_RED2_Pin);    //
Turns OFF
111     HAL_GPIO_TogglePin(LED_GREEN2_GPIO_Port,  LED_GREEN2_Pin);  //
```

```
Turns ON
112     HAL_Delay(5000);
113         /* Phase 4: Switch Direction 2 from GREEN to YELLOW */
114     HAL_GPIO_TogglePin(LED_GREEN2_GPIO_Port,  LED_GREEN2_Pin);  //
Turns OFF
115     HAL_GPIO_TogglePin(LED_YELLOW2_GPIO_Port, LED_YELLOW2_Pin); //
Turns ON
116     HAL_Delay(2000);
117         /* Back to Phase 1: Switch Direction 2 to RED, Direction 1
to GREEN */
118     HAL_GPIO_TogglePin(LED_YELLOW2_GPIO_Port, LED_YELLOW2_Pin); //
Turns OFF
119     HAL_GPIO_TogglePin(LED_RED2_GPIO_Port,    LED_RED2_Pin);    //
Turns ON
120     HAL_GPIO_TogglePin(LED_RED1_GPIO_Port,    LED_RED1_Pin);    //
Turns OFF
121     HAL_GPIO_TogglePin(LED_GREEN1_GPIO_Port,  LED_GREEN1_Pin);  //
Turns ON
122     HAL_Delay(5000);
123     /* USER CODE BEGIN 3 */
124   }
125   /* USER CODE END 3 */
```

6 Exercise 04: Seven-Segment Display

Lab 01 specifies a common-anode seven-segment display connected to PB0 - PB6. But while making this, I missed that, so I wired to PA1 - PA7 (This should yield the same results). The Proteus design and GPIO configuration use these pins for the seven segment outputs. The outputs are active-low, so a logic-low output turns a segment on. The `display7SEG` lookup table maps each integer from 0 to 9 to the corresponding segment pattern.

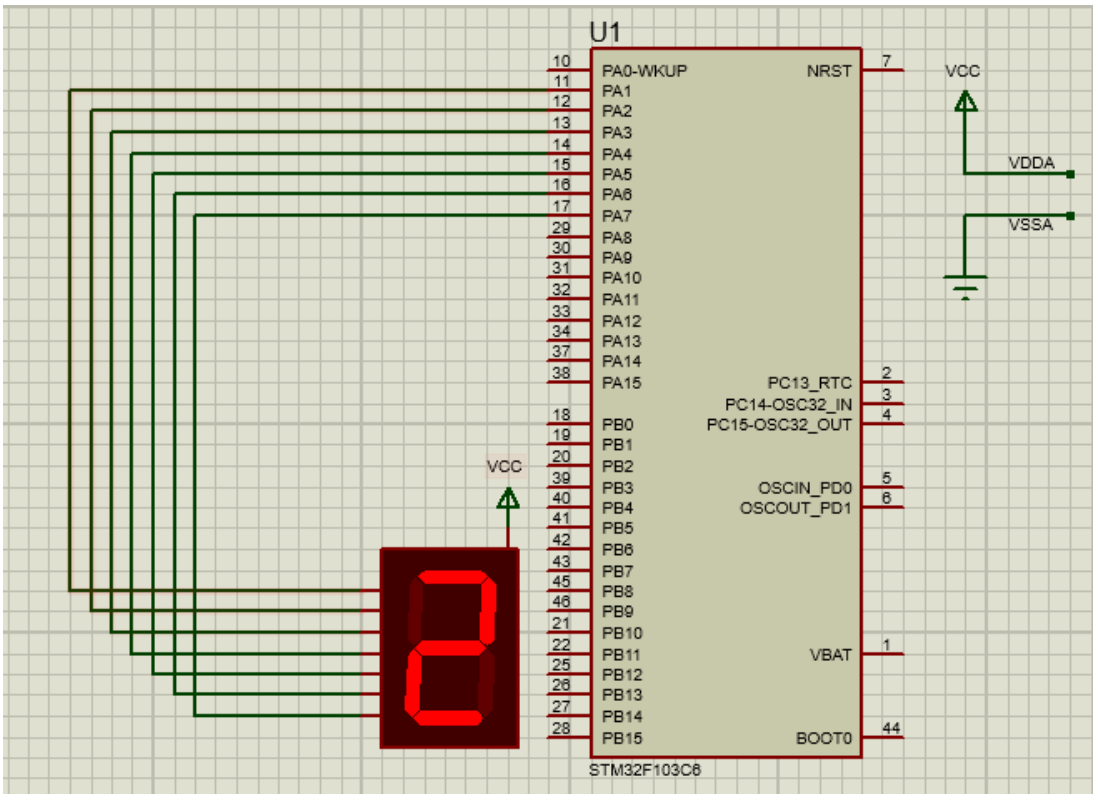


Figure 6.1: Exercise 04 seven-segment schematic and project files

As required by Lab 01, only the `display7SEG` function, with its integer parameter `num`, is shown.

Listing 6.1: Exercise 04 seven-segment display function

```
167 /* USER CODE BEGIN 4 */
168 void display7SEG(uint8_t num) {
169     uint8_t seg[10][7]=
170     {
171         {0,0,0,0,0,0,1}, //0
172         {1,0,0,1,1,1,1}, //1
173         {0,0,1,0,0,1,0}, //2
174         {0,0,0,0,1,1,0}, //3
175         {1,0,0,1,1,0,0}, //4
176         {0,1,0,0,1,0,0}, //5
177         {0,1,0,0,0,0,0}, //6
178         {0,0,0,1,1,1,1}, //7
179         {0,0,0,0,0,0,0}, //8
```

```
180         {0,0,0,0,1,0,0}  //9
181     };
182     HAL_GPIO_WritePin(a_GPIO_Port,a_Pin,seg[num][0]);
183     HAL_GPIO_WritePin(b_GPIO_Port,b_Pin,seg[num][1]);
184     HAL_GPIO_WritePin(c_GPIO_Port,c_Pin,seg[num][2]);
185     HAL_GPIO_WritePin(d_GPIO_Port,d_Pin,seg[num][3]);
186     HAL_GPIO_WritePin(e_GPIO_Port,e_Pin,seg[num][4]);
187     HAL_GPIO_WritePin(f_GPIO_Port,f_Pin,seg[num][5]);
188     HAL_GPIO_WritePin(g_GPIO_Port,g_Pin,seg[num][6]);
189 }
190 /* USER CODE END 4 */
```

7 Exercise 05: Traffic Countdown

The seven-segment display is integrated with the four-way traffic light to show the remaining time in each phase. The `display7SEG` function from Exercise 04 is reused through another helper function to keep everything somewhat readable.

Listing 7.1: Exercise 05 helper function

```
110 void countdownSEG(uint8_t num) {
111     for (int i = num; i >= 0; i--) {
112         display7SEG(i);
113         HAL_Delay(1000);
114     }
115 }
```

Listing 7.2: Exercise 05 traffic-light countdown loop

```
32   while (1)
33     {
34         //
   ------------------------------------------------------------------
35         // PHASE 1: Vertical GREEN (2 -> 0, 3s) | Horizontal RED
36         //
   ------------------------------------------------------------------
```

```
37          HAL_GPIO_WritePin(GREEN_LED_V_GPIO_Port, GREEN_LED_V_Pin,
    GPIO_PIN_RESET); // V GREEN ON
38          HAL_GPIO_WritePin(YELLOW_LED_V_GPIO_Port, YELLOW_LED_V_Pin,
    GPIO_PIN_SET);
39          HAL_GPIO_WritePin(RED_LED_V_GPIO_Port, RED_LED_V_Pin,
    GPIO_PIN_SET);
40          HAL_GPIO_WritePin(RED_LED_H_GPIO_Port, RED_LED_H_Pin,
    GPIO_PIN_RESET);      // H RED ON
41          HAL_GPIO_WritePin(YELLOW_LED_H_GPIO_Port, YELLOW_LED_H_Pin,
    GPIO_PIN_SET);
42          HAL_GPIO_WritePin(GREEN_LED_H_GPIO_Port, GREEN_LED_H_Pin,
    GPIO_PIN_SET);
43          countdownSEG(2); // Counts 2, 1, 0 (3 seconds)
44          //
    ------------------------------------------------------------------
45          // PHASE 2: Vertical YELLOW (1 -> 0, 2s) | Horizontal RED
46          //
    ------------------------------------------------------------------
47          HAL_GPIO_WritePin(GREEN_LED_V_GPIO_Port, GREEN_LED_V_Pin,
    GPIO_PIN_SET);
48          HAL_GPIO_WritePin(YELLOW_LED_V_GPIO_Port, YELLOW_LED_V_Pin,
    GPIO_PIN_RESET); // V YELLOW ON
49          countdownSEG(1); // Counts 1, 0 (2 seconds)
50          //
    ------------------------------------------------------------------
51          // PHASE 3: Vertical RED (4 -> 0, 5s) | Horizontal GREEN (3s
    ) -> YELLOW (2s)
52          //
    ------------------------------------------------------------------
53          HAL_GPIO_WritePin(YELLOW_LED_V_GPIO_Port, YELLOW_LED_V_Pin,
    GPIO_PIN_SET);
54          HAL_GPIO_WritePin(RED_LED_V_GPIO_Port, RED_LED_V_Pin,
    GPIO_PIN_RESET);      // V RED ON
55          HAL_GPIO_WritePin(RED_LED_H_GPIO_Port, RED_LED_H_Pin,
    GPIO_PIN_SET);        // H RED OFF
```

```
56          for (int i = 4; i >= 0; i--)
57          {
58              if (i >= 2) // Counts 4, 3, 2 (3s): Horizontal GREEN
59              {
60                  HAL_GPIO_WritePin(GREEN_LED_H_GPIO_Port,
    GREEN_LED_H_Pin, GPIO_PIN_RESET); // H GREEN ON
61                  HAL_GPIO_WritePin(YELLOW_LED_H_GPIO_Port,
    YELLOW_LED_H_Pin, GPIO_PIN_SET);
62              }
63              else // Counts 1, 0 (2s): Horizontal YELLOW
64              {
65                  HAL_GPIO_WritePin(GREEN_LED_H_GPIO_Port,
    GREEN_LED_H_Pin, GPIO_PIN_SET);
66                  HAL_GPIO_WritePin(YELLOW_LED_H_GPIO_Port,
    YELLOW_LED_H_Pin, GPIO_PIN_RESET); // H YELLOW ON
67              }
68              display7SEG(i);
69              HAL_Delay(1000);
70          }
71      }
72    /* USER CODE END 3 */
```

8 Exercise 06: Analog Clock LED Test

Twelve LEDs representing the clock positions are connected to PA4 - PA15. The test turns on each LED in sequence, waits 500 ms, and then turns it off before moving to the next position.

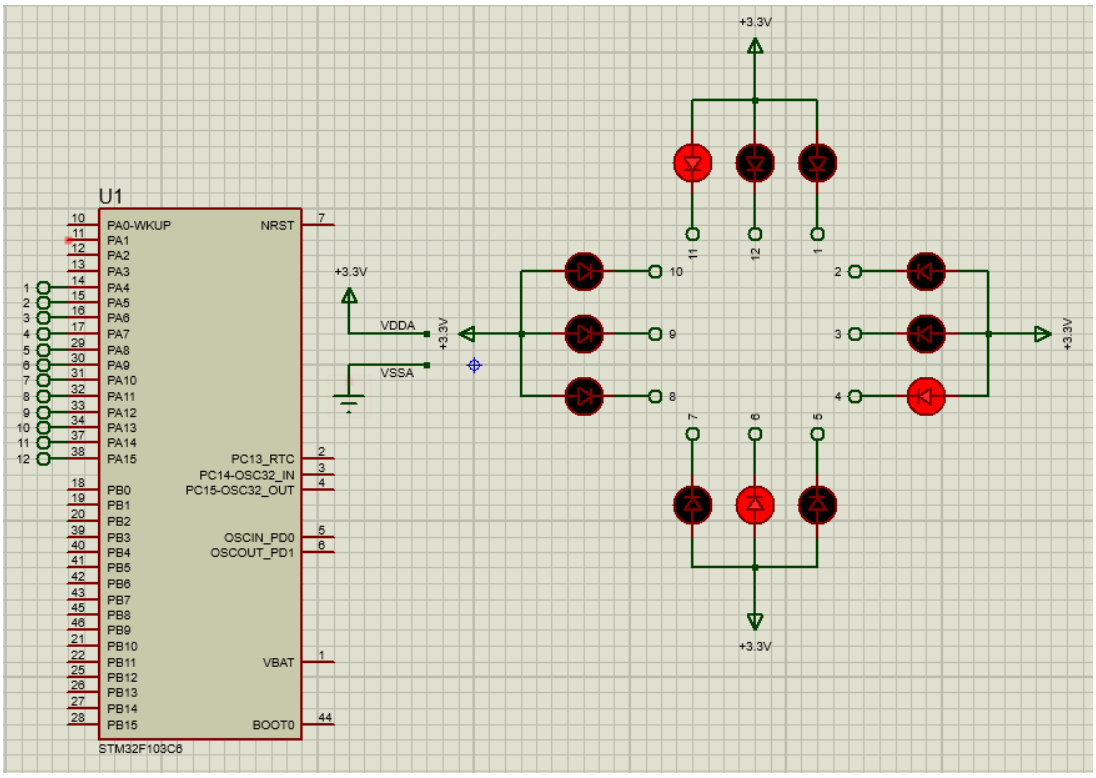


Figure 8.1: Exercise 06 twelve-LED clock schematic and project files

Listing 8.1: Exercise 06 sequential LED test

9 Exercise 07: Clear All Clock LEDs

The `clearAllClock` function turns off all twelve active-low LEDs with one HAL GPIO call.

Listing 9.1: Exercise 07 clearAllClock function

```
65 void clearAllClocks(void) {
66     HAL_GPIO_WritePin(GPIOA, 0xFFF0, GPIO_PIN_SET);
67 }
```

10 Exercise 08: Set a Clock Position

The `setNumberOnClock` function accepts a value from 0 to 11 and turns on the corresponding LED.

Listing 10.1: Exercise 08 setNumberOnClock function

```
69 void setNumberOnClock(int num) {
70     if (num >= 0 && num <= 11) {
71         // 0 -> PA15, 1 -> PA4, 2 -> PA5 ... 11 -> PA14
72         int pin = (num == 0) ? 15 : (3 + num);
73         HAL_GPIO_WritePin(GPIOA, (1 << pin), GPIO_PIN_RESET); // ON (
   Active-LOW)
74     }
75 }
```

11 Exercise 09: Clear a Clock Position

The `clearNumberOnClock` function accepts a value from 0 to 11 and turns off the corresponding LED. This is just inverting `setNumberOnClock`.

Listing 11.1: Exercise 09 clearNumberOnClock function

```
77 void clearNumberOnClock(int num) {
78     if (num >= 0 && num <= 11) {
79         // 0 -> PA15, 1 -> PA4, 2 -> PA5 ... 11 -> PA14
80         int pin = (num == 0) ? 15 : (3 + num);
81         HAL_GPIO_WritePin(GPIOA, (1 << pin), GPIO_PIN_SET); // OFF (
   Active-LOW)
82     }
83 }
```

12 Exercise 10: Integrated Analog Clock

The final implementation clears the previous state and illuminates the clock positions representing the hour, minute, and second values. The functions from Exercises 07 - 09 are reused and are not repeated below. Then we use 100ms for 1 real life second for testing purposes.

Listing 12.1: Exercise 10 clock update loop

```
30   while (1) {
31       // 1. Clear display
```

```
32          clearAllClocks();
33          // 2. Map 60-second and 60-minute values to 12 LED positions
    (0 to 11)
34          int sec_pos = sec / 5;
35          int min_pos = min / 5;
36          int hour_pos = hour;
37          // 3. Light up hands (overlapping hands naturally combine)
38          setNumberOnClock(sec_pos);
39          setNumberOnClock(min_pos);
40          setNumberOnClock(hour_pos);
41          // 4. Tick speed (100ms per second)
42          HAL_Delay(100);
43          // 5. Advance clock time
44          sec++;
45          if (sec >= 60) {
46              sec = 0;
47              min++;
48              if (min >= 60) {
49                  min = 0;
50                  hour = hour % 12;
51              }
52          }
53      }
54      /* USER CODE END 3 */
55  }
```

13 Conclusion

The exercises demonstrate active-low GPIO control, timed traffic-light state transitions, seven-segment decoding, countdown integration, and an LED-based analog clock. The report follows the Lab 01 deliverables and includes only the implementation portions relevant to each exercise.